

Relatório Técnico Final — DAMD Sistema de Agendamento de Transporte

Disciplina: Laboratório de Desenvolvimento de Aplicações Móveis e Distribuídas (LDAMD)

Instituição: PUC Minas — Engenharia de Software, 5º Período

Semestre: 1º Semestre 2026

Aluno: Thiago Branco **Vídeo de demonstração (screencast):** <https://www.youtube.com/watch?v=VylVB6A0p5Q>

1. Introdução e Domínio

O DAMD é um sistema distribuído de agendamento de transporte que conecta passageiros e motoristas por meio de uma arquitetura orientada a eventos. O domínio segue a estrutura cliente/prestador exigida pelo projeto: o passageiro (cliente) solicita corridas, e o motorista (prestador) recebe as demandas, aceita e executa o serviço.

O sistema foi desenvolvido em quatro sprints incrementais ao longo do semestre, culminando neste relatório que descreve a arquitetura completa implementada.

1.1 Perfis de usuário

- **Passageiro:** solicita corridas, acompanha o status em tempo real e recebe notificações de atualização.
- **Motorista:** visualiza corridas disponíveis, aceita demandas e atualiza o progresso (aceitar → iniciar → concluir).

1.2 Fluxo principal

```
Passageiro cria corrida (PENDING)
  → backend publica ride.created no MOM
  → consumer:driver persiste notificação de demanda
  → app do motorista é notificado (polling) → aceita (ACCEPTED + driverId)
  → backend publica ride.status_updated no MOM
  → consumer:passenger persiste notificação para o passageiro
  → app do passageiro reflete a mudança automaticamente
  → motorista inicia (IN_PROGRESS) e conclui (COMPLETED) → passageiro acompanha a cada etapa
```

2. Arquitetura Implementada

2.1 Visão geral dos componentes

O sistema é composto por quatro componentes independentes que se comunicam por meio de protocolo HTTP (REST) e mensagens assíncronas (AMQP/RabbitMQ):

Componente	Tecnologia	Função
Backend REST	Node.js 20 + Express + TypeScript	API REST + publicação de eventos no MOM
Banco de dados	PostgreSQL 16 (Docker)	Persistência de entidades e notificações
MOM	RabbitMQ 3.13 (Docker)	Exchange topic + filas de eventos
Consumer do motorista	Node.js (processo separado)	Consome <code>ride.created</code> , persiste <code>DriverNotification</code>
Consumer do passageiro	Node.js (processo separado)	Consome <code>ride.status_updated</code> , persiste <code>Notification</code>
App do passageiro	Flutter/Dart (<code>mobile/</code>)	Interface do cliente; polling de notificações
App do motorista	Flutter/Dart (<code>mobile-driver/</code>)	Interface do prestador; polling de demandas

2.2 Arquitetura Orientada a Eventos (EDA)

A comunicação assíncrona central do sistema é implementada via RabbitMQ com o padrão **Publish/Subscribe** sobre um exchange do tipo `topic` (HOHPE; WOOLF, 2003):

- **Exchange:** `rides.events` (durable, topic)
- **Filas:**
 - `notifications.driver` — vinculada à routing key `ride.created`
 - `notifications.passenger` — vinculada à routing key `ride.status_updated`

O backend publica eventos em formato de envelope padronizado:

```
{
  "event": "ride.created",
  "occurredAt": "2026-06-28T14:30:00.000Z",
```

```
"data": { ... }  
}
```

A publicação é **fire-and-forget**: erros do broker são capturados com try/catch e logados, mas nunca falham a requisição REST. A persistência no banco é a fonte de verdade do sistema. Esta decisão garante que a latência do broker não impacte a experiência do usuário (RICHARDSON, 2018).

2.3 Clean Architecture no backend

O backend segue os princípios da Clean Architecture (MARTIN, 2019): as dependências sempre apontam para dentro, das camadas externas para as internas.

domain/entities/	← Entidades puras (Ride, User, Driver, Notification, DriverNotification)
application/	
repositories/	← Interfaces (ports) – o domínio não conhece Prisma
use-cases/	← Regras de negócio puras
infrastructure/	
database/repositories/	← Implementações Prisma dos ports
http/	← Controllers e rotas Express
messaging/	← RabbitMQ publisher, handlers, consumer-runner

A injeção de dependência é manual: os controllers instanciam repositórios concretos e use cases no topo do arquivo. Não há container de DI — escolha deliberada para manter o código simples e transparente para fins didáticos.

2.4 Clean Architecture nos apps Flutter

Ambos os apps (`mobile/` e `mobile-driver/`) seguem a mesma filosofia de camadas:

domain/	← Entidades e interfaces de repositório (sem Flutter, sem Dio)
data/	← Implementações com Dio, DTOs, serialização JSON
application/	← Estado com Riverpod (AsyncNotifier/FutureProvider), DI via providers
presentation/	← Telas e widgets Flutter; roteamento com GoRouter

A regra de dependência é: `presentation → application → domain` e `data → domain`. O `domain` não importa nenhum pacote externo, o que torna as entidades e contratos independentes de framework — princípio fundamental da Clean Architecture (MARTIN, 2019).

2.5 Modelo de dados

```
User      ← passageiro
Driver    ← motorista (com vehicleModel, licensePlate)
Ride      ← corrida (FK userId, FK opcional driverId, enum RideStatus)
Notification ← notificação para o passageiro (userId, rideId, type, read)
DriverNotification ← demanda broadcast para motoristas (rideId, read)
```

A máquina de estados das corridas (`VALID_TRANSITIONS`) vive no use case `UpdateRideStatusUseCase` e garante que apenas transições válidas sejam aceitas: `PENDING → ACCEPTED/CANCELLED` , `ACCEPTED → IN_PROGRESS/CANCELLED` , `IN_PROGRESS → COMPLETED/CANCELLED` .

3. Fluxo de Eventos (Catálogo)

Evento `ride.created`

Campo	Valor
Routing key	<code>ride.created</code>
Exchange	<code>rides.events</code> (topic)
Produtor	<code>CreateRideUseCase</code> → <code>RabbitMQEventPublisher</code>
Fila	<code>notifications.driver</code>
Consumidor	<code>consumer:driver</code> → <code>notifyDriverHandler</code>
Efeito	Persiste <code>DriverNotification</code> (broadcast); app do motorista detecta via polling

Payload de exemplo:

```
{
  "event": "ride.created",
  "occurredAt": "2026-06-28T14:30:00.000Z",
  "data": {
    "rideId": "uuid-da-corrida",
    "userId": "uuid-do-passageiro",
    "origin": "Praça da Liberdade, BH",
    "destination": "Aeroporto de Confins",
    "scheduledAt": "2026-06-29T08:00:00.000Z"
  }
}
```

Evento `ride.status_updated`

Campo	Valor
Routing key	<code>ride.status_updated</code>
Exchange	<code>rides.events</code> (topic)
Produtor	<code>UpdateRideStatusUseCase</code> → <code>RabbitMQEventPublisher</code>
Fila	<code>notifications.passenger</code>
Consumidor	<code>consumer:passenger</code> → <code>notifyPassengerHandler</code>
Efeito	Persiste <code>Notification</code> para o passageiro; app detecta via polling

Payload de exemplo:

```
{
  "event": "ride.status_updated",
  "occurredAt": "2026-06-28T14:35:00.000Z",
  "data": {
    "rideId": "uuid-da-corrida",
    "userId": "uuid-do-passageiro",
    "driverId": "uuid-do-motorista",
    "previousStatus": "PENDING",
    "newStatus": "ACCEPTED"
  }
}
```

4. Decisões de Design

4.1 Notificações persistidas + polling (não push real)

A atualização assíncrona nos dois apps é implementada via **polling sobre tabela alimentada pelo MOM**, e não por WebSocket ou FCM. Esta escolha foi deliberada:

- **Demonstrabilidade:** o evento do MOM tem efeito visível e verificável (linha na tabela `DriverNotification` / `Notification`), tornando a assincronicidade auditável.
- **Simplicidade:** o app faz uma chamada GET periódica (5s) a um endpoint barato, sem gerenciar conexões persistentes.

- **Confiabilidade:** `await handler()` antes do `channel.ack(msg)` garante que, se o banco falhar, a mensagem não é confirmada e pode ser reprocessada. `nack(requeue:false)` descarta mensagens com falha de banco sem requeue, evitando loops infinitos.

O padrão é simétrico nos dois sentidos: `ride.created` → `DriverNotification` → app do motorista;
`ride.status_updated` → `Notification` → app do passageiro.

4.2 DriverNotification como broadcast

Como uma corrida nova (status `PENDING`) ainda não tem motorista associado, não é possível endereçar a notificação a um `driverId` específico. A `DriverNotification` é broadcast — qualquer motorista pode ver e aceitar. Isso é uma simplificação adequada para um sistema de demonstração com um motorista. Em produção, um sistema de matching/despacho tomaria essa decisão e enviaria a notificação apenas ao motorista selecionado.

4.3 "Recusar" é client-side

O backend não tem estado "recusado" — a corrida deve permanecer `PENDING` para outros motoristas poderem aceitá-la. Ao recusar, o app apenas remove o item da visão local e atualiza a lista. A máquina de estados já garante que um segundo aceite (`PENDING` → `ACCEPTED`) só ocorre uma vez; tentativas subsequentes recebem erro 400.

4.4 driverId no aceite

O `PATCH /rides/:id/status` aceita `driverId` opcional no body. Quando presente e a transição é `PENDING` → `ACCEPTED`, o `driverId` é gravado na corrida e propagado no payload do evento `ride.status_updated`. Isso foi resolvido na Sprint 3 para que o app do passageiro pudesse exibir informações do motorista e para que o fluxo de Sprint 4 ficasse mais limpo.

4.5 Identidade sem autenticação

O sistema não implementa autenticação (fora do escopo da disciplina). A identidade do usuário/motorista é mantida por `userId` / `driverId` persistidos localmente via `shared_preferences`. A primeira tela de cada app é uma tela de identificação leve (`POST /users` ou `POST /drivers`) que cria ou recupera o perfil.

5. Dificuldades e Soluções

5.1 Emulador Android × localhost

O emulador Android não consegue acessar `localhost` do host diretamente. A solução é usar o IP especial `10.0.2.2`, que o emulador mapeia para o host. Para dispositivos físicos ou IPs customizados, o `baseUrl` pode ser sobrescrito via `--dart-define=API_BASE_URL=`.

5.2 Sincronização do ack com a persistência

Na Sprint 2, o `consumer-runner` dava `ack` imediatamente, antes do handler completar. Se o handler assíncrono (que persiste no banco) falhasse, a mensagem era confirmada e a notificação se perdia. A solução foi mudar a assinatura do handler para `() => void | Promise<void>` e fazer `await handler(envelope)` antes do `channel.ack(msg)`. Falhas de banco agora resultam em `nack(requeue:false)`.

5.3 Estado terminal no polling

O polling (5s) no detalhe da corrida do app do passageiro para quando o status atinge um estado terminal (`COMPLETED` ou `CANCELLED`), evitando chamadas desnecessárias à API após o fim do ciclo de vida da corrida.

5.4 Race condition no aceite

Se dois motoristas tentarem aceitar a mesma corrida simultaneamente, o segundo aceite é barrado pela máquina de estados do backend (a transição `PENDING → ACCEPTED` só é válida uma vez; o segundo tenta `ACCEPTED → ACCEPTED`, que não é uma transição permitida). O app do motorista trata o erro 400 exibindo "Corrida não está mais disponível".

6. Reflexão sobre os Padrões Estudados

6.1 Event-Driven Architecture (EDA)

A EDA foi o princípio central do projeto desde a Sprint 2. Em vez de chamadas REST síncronas entre serviços, o sistema usa eventos para comunicar mudanças de estado. Isso garante **desacoplamento temporal** (os consumers processam mensagens de forma assíncrona e independente) e **desacoplamento espacial** (produtor e consumidor não precisam estar disponíveis ao mesmo tempo) — conforme descrito em COULOURIS et al. (2011) no conceito de comunicação indireta por espaço e tempo.

6.2 Middleware Orientado a Mensagens (MOM)

O RabbitMQ implementa o padrão **Publish-Subscribe** com exchange do tipo `topic`, conforme os Enterprise Integration Patterns de HOHPE e WOOLF (2003). O uso de routing keys (`ride.created`,

`ride.status_updated`) permite que múltiplos consumidores sejam adicionados ao mesmo fluxo sem alterar o produtor — princípio Open/Closed aplicado a sistemas de mensageria.

O padrão **Message Envelope** (envelope com `event`, `occurredAt` e `data`) foi adotado para padronizar o contrato entre produtor e consumidores, facilitando a adição de novos tipos de evento sem quebrar consumidores existentes.

6.3 Clean Architecture

A organização em camadas com regra de dependência unidirecional (MARTIN, 2019) permitiu que as sprints fossem incrementais sem quebrar o que já funcionava. As entidades de domínio e os ports (interfaces de repositório) nunca mudaram de assinatura; apenas novos casos de uso e implementações de infraestrutura foram adicionados. No backend, o mesmo princípio facilitou a troca do handler do motorista (de stub para implementação real) sem alterar nenhum use case ou entidade existente.

6.4 REST e Microservices

O backend segue os princípios REST para exposição de recursos (RICHARDSON, 2018), com endpoints organizados por recurso (`/rides`, `/users`, `/drivers`, `/notifications`, `/driver/notifications`) e uso semântico dos métodos HTTP (POST para criação, GET para listagem, PATCH para atualização parcial de status). A máquina de estados das corridas implementa o padrão de estado finito descrito por Richardson para transições de recursos RESTful.

7. Conclusão

O sistema DAMD demonstra com sucesso a integração de todos os conceitos estudados na disciplina: uma arquitetura orientada a eventos com RabbitMQ como MOM, um backend REST com Clean Architecture em Node.js/TypeScript, dois aplicativos móveis Flutter com Riverpod para gerenciamento de estado, e um fluxo ponta-a-ponta que evidencia a assincronicidade real via eventos (não polling cego).

O "money shot" — o app do passageiro refletindo a aceitação da corrida pelo motorista sem nenhuma ação manual, com o gatilho real sendo um evento `ride.status_updated` no RabbitMQ — demonstra que os quatro componentes funcionam de forma integrada e distribuída conforme proposto.

O vídeo de demonstração completo do fluxo ponta-a-ponta está disponível em: <https://www.youtube.com/watch?v=VylVB6A0p5Q>

Referências Bibliográficas

MARTIN, Robert C. **Arquitetura limpa: o guia do artesão para estrutura e design de software**. Rio de Janeiro: Alta Books, 2019. (Fundamenta os princípios de Clean Architecture adotados na organização dos apps Flutter e do backend.)

HOHPE, Gregor; WOOLF, Bobby. **Enterprise Integration Patterns: designing, building, and deploying messaging solutions**. Boston: Addison-Wesley, 2003. (Padrões de integração por mensagens: filas, tópicos, pub/sub, envelope de mensagem. Base teórica para o MOM com RabbitMQ.)

RICHARDSON, Chris. **Microservices patterns: with examples in Java**. Shelter Island: Manning, 2018. (Padrões de EDA, comunicação assíncrona entre serviços, máquina de estados de recursos REST.)

COULOURIS, George et al. **Distributed Systems: concepts and design**. 5th ed. Boston: Addison-Wesley, 2011. (Conceitos de sistemas distribuídos, comunicação indireta por espaço e tempo, middlewares orientados a mensagens.)

BAILEY, Thomas. **Flutter for beginners**. 3rd ed. Birmingham: Packt, 2023. (Referência para o desenvolvimento dos aplicativos móveis com Flutter 3.x e Dart 3.x.)